Phase 3: 训练组件详解

1. Cross-Entropy Loss

1.1 从信息论到 Loss 函数

交叉熵的含义：度量"模型预测分布 p_θ 与真实分布 p_data 之间的差异"。

对于语言模型，真实分布在训练数据中是 one-hot 的（下一个 token 已知），所以交叉熵简化为负对数似然：

```
loss = -log p_θ(x_{i+1} | x_{1:i})
     = -log softmax(o_i)[x_{i+1}]
```

其中 o_i ∈ R^V 是模型在位置 i 输出的 logits 向量。

1.2 数值稳定的展开

直接计算 `softmax` 再取 `log` 会有精度问题。展开后可以大量简化：

```
-log softmax(o)[target]
= -log ( exp(o[target]) / Σ_j exp(o[j]) )
= -(o[target] - log Σ_j exp(o[j]))
= -o[target] + log Σ_j exp(o[j])
```

再用 log-sum-exp trick（减去最大值防溢出）：

```
max_o = max(o)
log Σ_j exp(o[j]) = max_o + log Σ_j exp(o[j] - max_o)
```

最终：

```
loss = -(o[target] - max_o) + log Σ_j exp(o[j] - max_o)
```

关键点：不需要先算 softmax 再取 log——直接从 logits 一步到 loss，更稳定也更高效。

1.3 实现要点

输入形状：

- logits: `(batch_size, vocab_size)` 或 `(batch_size, seq_len, vocab_size)`
- targets: `(batch_size,)` 或 `(batch_size, seq_len)`

返回值：所有位置的平均 loss（scalar）

思路：

1. 沿 vocab_size 维度取 max
2. logits 减去 max
3. 用 `gather` 或高级索引取出 target 位置的 logit
4. 计算 log-sum-exp
5. loss = -target_logit + log_sum_exp
6. 取 mean

1.4 Perplexity

```
perplexity = exp(average_loss)
```

- perplexity = 1：模型完美预测每个 token
- perplexity = V：模型和随机猜一样（均匀分布）
- perplexity 可以理解为"模型在每一步平均犹豫的选项数"

2. AdamW 优化器

2.1 从 SGD 到 Adam

SGD：

```
θ_{t+1} = θ_t - α · g_t
```

问题：所有参数用同一个学习率；噪声大时不稳定。

Adam（Adaptive Moment Estimation）：

```
m_t = β₁ · m_{t-1} + (1 - β₁) · g_t          # 一阶矩估计（梯度的指数移动平均）
v_t = β₂ · v_{t-1} + (1 - β₂) · g_t²          # 二阶矩估计（梯度平方的指数移动平均）
m̂_t = m_t / (1 - β₁^t)                        # 偏差校正
v̂_t = v_t / (1 - β₂^t)                        # 偏差校正
θ_{t+1} = θ_t - α · m̂_t / (√v̂_t + ε)
```

直觉：

- m_t 是平滑的梯度方向（动量），减少噪声
- v_t 是梯度大小的估计，用于自适应调整步长
- 梯度大的参数步长小（除以大的 √v），梯度小的参数步长大
- 偏差校正：初始 $m_0 = v_0 = 0$ 会导致初期估计偏低，校正项补偿这个偏差

2.2 AdamW vs Adam + L2

Adam + L2 正则化：

```
g_t = ∇L(θ_t) + λ·θ_t      ← weight decay 混入梯度
m_t, v_t 根据这个 g_t 更新
```

问题：weight decay 被 Adam 的自适应步长缩放了，效果与 SGD+L2 不同。

AdamW（解耦的 weight decay）：

```
g_t = ∇L(θ_t)               ← 纯梯度
m_t, v_t 根据纯梯度更新
θ_{t+1} = θ_t - α · (m̂_t / (√v̂_t + ε) + λ·θ_t)
```

或等价地：

```
θ_{t+1} = (1 - α·λ) · θ_t - α · m̂_t / (√v̂_t + ε)
```

weight decay 独立于梯度的自适应缩放，更符合直觉。

2.3 AdamW 伪代码（作业要求的精确版本）

```
初始化:
  对每个参数 θ:
    m₀ = 0  (与 θ 同形状)
    v₀ = 0  (与 θ 同形状)
    t = 0

每步更新:
  t += 1
  g = θ.grad

  m = β₁ · m + (1 - β₁) · g
  v = β₂ · v + (1 - β₂) · g²

  m̂ = m / (1 - β₁^t)
  v̂ = v / (1 - β₂^t)

  θ = θ - lr · (m̂ / (√v̂ + ε) + λ · θ)
```

2.4 实现要点

继承 `torch.optim.Optimizer`：

```python
class AdamW(torch.optim.Optimizer):
    def __init__(self, params, lr, betas, eps, weight_decay):
        defaults = dict(lr=lr, betas=betas, eps=eps, weight_decay=weight_decay)
        super().__init__(params, defaults)

    def step(self):
        for group in self.param_groups:
            for p in group['params']:
                if p.grad is None:
                    continue
                # 从 self.state[p] 读取/初始化 m, v, step
                # 执行更新
                # p.data 原地修改
```

状态管理：

- `self.state` 是一个 defaultdict，以参数对象为 key

- 第一次访问某参数时需要初始化 m=0, v=0, step=0
- 之后每次 step 递增

内存影响：

- 每个参数需要额外存储 m 和 v（都与参数同形状）
- 总内存：参数 × 3（原始参数 + m + v）
- 加上梯度：参数 × 4

2.5 常用超参数

超参数	典型值	说明
lr	1e-4 ~ 1e-3	需要 tuning
β_1	0.9	一阶矩衰减率
β_2	0.95 ~ 0.999	二阶矩衰减率（大模型常用 0.95）
ε	1e-8	防止除零
λ (weight_decay)	0.01 ~ 0.1	正则化强度

3. Cosine Learning Rate Schedule with Warmup

3.1 为什么需要 LR Schedule

- 训练初期：参数还是随机的，大学习率可能导致不稳定
- 训练中期：需要较大学习率快速下降 loss
- 训练后期：需要小学习率精细调整

3.2 三段式调度

```
给定:
  t          — 当前步数
  α_max      — 最大学习率
  α_min      — 最小学习率
  T_w        — warmup 步数
  T_c        — cosine 周期步数

学习率:
  if t < T_w:
    α_t = (t / T_w) × α_max              # 线性 warmup

  elif T_w ≤ t ≤ T_c:
    α_t = α_min + ½(α_max - α_min)(1 + cos(π × (t - T_w)/(T_c - T_w)))
                                          # cosine 衰减

  else:  # t > T_c
    α_t = α_min                           # 保持最小值
```

3.3 图形直觉

```
α_max ──────╮
            |╲
            |  ╲     cosine 曲线
            |    ╲
            |      ╲
    ╱       |        ╲
   ╱ warmup |          ╲──── α_min
  ╱         |
──┴─────────┴──────────────────── t
  0       T_w              T_c
```

3.4 实现要点

- 纯数学函数，不需要维护状态
- 在训练循环中，每步调用以获取当前 lr
- 然后手动设置优化器的 lr： `for group in optimizer.param_groups: group['lr'] = new_lr`
- 或者直接在 AdamW.step 内部根据传入的 lr 更新

4. Gradient Clipping

4.1 为什么需要

训练 Transformer 时，某些 batch 可能产生异常大的梯度（"gradient spike"），导致：

- 参数瞬间跳到很差的位置
- 优化器状态被污染
- 训练不可逆转地变差

梯度裁剪（gradient clipping）是最简单有效的防护措施。

4.2 全局 L2 范数裁剪

```
1. 计算全局梯度 L2 范数:
   total_norm = √(Σ_p Σ_i (p.grad_i)²)

   即把所有参数的梯度拉平成一个大向量，算其 L2 范数

2. 如果 total_norm > M:
   clip_coef = M / (total_norm + ε)      # ε = 1e-6 防除零
   对每个参数: p.grad *= clip_coef
```

效果：梯度方向不变，只缩放大小。

4.3 实现要点

- 遍历所有参数，跳过 `grad is None` 的
- 计算每个参数梯度的范数平方（`(p.grad ** 2).sum()`），然后求总和再开根号
- 或者用 `torch.norm` / `torch.linalg.norm`
- 裁剪是原地操作：`p.grad.mul_(clip_coef)` 或 `p.grad *= clip_coef`

5. Data Loading (get_batch)

5.1 语言模型的数据组织

训练数据是一长串 token IDs：$[x_1, x_2, x_3, ..., x_N]$

每个训练样本：

- 输入： `[x_i, x_{i+1}, ..., x_{i+m-1}]` 长度为 context_length
- 标签： `[x_{i+1}, x_{i+2}, ..., x_{i+m}]` 长度为 context_length（即输入右移一位）

5.2 采样策略

```
dataset: [x₁, x₂, x₃, ..., x_N]      N 个 token

对于 batch_size=B, context_length=m:
  随机采样 B 个起始位置 i₁, i₂, ..., i_B
  其中 i_k ∈ [0, N - m - 1]    (确保有 m+1 个 token 够用)

  inputs[k] = dataset[i_k : i_k + m]
  labels[k] = dataset[i_k + 1 : i_k + m + 1]
```

5.3 实现要点

- 用 `np.random.randint` 或 `torch.randint` 采样起始位置
- 从 numpy 数组切片，然后转为 `torch.LongTensor`
- 放到指定 device： `.to(device)`
- 大数据集用 `np.memmap` 加载，避免全部读入内存

6. Checkpointing

6.1 保存什么

一个 checkpoint 需要包含：

1. **模型参数**： `model.state_dict()`
2. **优化器状态**： `optimizer.state_dict()` （包含 m, v, step 等）
3. **当前迭代步数**：用于恢复训练时知道从哪步继续

6.2 PyTorch 序列化

```python
# 保存
checkpoint = {
    'model': model.state_dict(),
    'optimizer': optimizer.state_dict(),
    'iteration': iteration,
}
torch.save(checkpoint, path_or_file)

# 加载
checkpoint = torch.load(path_or_file)
model.load_state_dict(checkpoint['model'])
optimizer.load_state_dict(checkpoint['optimizer'])
iteration = checkpoint['iteration']
```

6.3 注意事项

- `torch.save` / `torch.load` 支持路径字符串和 file-like 对象（如 `io.BytesIO`）
- 加载时可能需要指定 `map_location`（如从 GPU checkpoint 加载到 CPU）
- `optimizer.state_dict()` 包含了所有参数的 m, v 等状态，确保恢复后优化器行为一致

7. 文本生成（Decoding）

7.1 自回归生成流程

```
给定 prompt tokens: [t₁, t₂, ..., t_k]

循环:
  1. 把当前序列输入模型
  2. 取最后一个位置的 logits: logits[-1] ∈ R^V
  3. 应用 temperature scaling 和 top-p
  4. 采样下一个 token
  5. 追加到序列
  6. 如果生成了 <|endoftext|> 或达到最大长度 → 停止
```

7.2 Temperature Scaling

```
softmax(v, τ)_i = exp(v_i / τ) / Σ_j exp(v_j / τ)
```

- τ → 0：分布变成 one-hot（贪心选最大的）
- τ = 1：正常 softmax
- τ → ∞：分布变成均匀分布（完全随机）

实现：在 softmax 前把 logits 除以 temperature 即可。

7.3 Top-p (Nucleus) Sampling

```
1. 对 logits 做 softmax 得到概率分布 q
2. 按概率从大到小排序
3. 累加概率，找到最小的集合 V(p) 使得 Σ_{j∈V(p)} q_j ≥ p
4. 把不在 V(p) 中的 token 概率置为 0
5. 重新归一化
6. 从归一化后的分布中采样
```

效果：只从"最有可能"的 token 中采样，避免选到概率极低的"奇怪" token。

7.4 实现提示

- `torch.sort` 按概率降序排列
- `torch.cumsum` 计算累积概率
- 找到 cumsum > p 的位置，把这些位置对应的概率设为 0（或 logits 设为 -inf）
- `torch.multinomial` 从概率分布中采样